Python Code

```
import { Request, Response } from "express";
import Order from "../models/order";
import Restaurant from "../models/restaurant";
import User from "../models/user";

const getAnalyticsData = async (req: Request, res: Response) => {
  try {
    console.log("Analytics API called with timeRange:", req.query.timeRange);

    // Get time range from query params (default to 30 days)
    const timeRange = req.query.timeRange || "30d";
    const days =
      timeRange === "7d"
        ? 7
        : timeRange === "90d"
        ? 90
        : timeRange === "1y"
        ? 365
        : 30;

    const startDate = new Date();
    startDate.setDate(startDate.getDate() - days);

    console.log("Fetching orders from:", startDate.toISOString());

    // Get all orders within the time range
    const orders = await Order.find({
      createdAt: { $gte: startDate },
    })
      .populate("restaurant")
      .populate("user");

    console.log("Found orders:", orders.length);
    console.log("Orders count:", orders.length);

    // Calculate total orders
    const totalOrders = orders.length;

    // Calculate total revenue (sum of totalAmount)
    const totalRevenue = orders.reduce((sum: number, order: any) => {
      const amount = order.totalAmount || 0;
      return sum + amount;
    }, 0);

    // Calculate average order value
    const averageOrderValue = totalOrders > 0 ? totalRevenue / totalOrders : 0;

    console.log("Total revenue:", totalRevenue);
    console.log("Average order value:", averageOrderValue);

```

```
    // Get unique customers
    const uniqueCustomers = new Set(
      orders
        .filter((order: any) => order.user)
        .map((order: any) => order.user.toString())
    ).size;

    console.log("Unique customers:", uniqueCustomers);

    // Get top cities by orders
    const cityStats = orders
      .filter(
        (order: any) => order.deliveryDetails && order.deliveryDetails.city
      )
      .reduce((acc, order: any) => {
        const city = order.deliveryDetails.city;
        if (!acc[city]) {
          acc[city] = { orders: 0, revenue: 0 };
        }
        acc[city].orders += 1;
        acc[city].revenue += order.totalAmount || 0;
        return acc;
      }, {} as Record<string, { orders: number; revenue: number }>);

    const topCities = Object.entries(cityStats)
      .map(([city, stats]) => ({
        city,
        orders: stats.orders,
        revenue: stats.revenue,
      }))
      .sort((a, b) => b.orders - a.orders)
      .slice(0, 5);

    console.log("Top cities:", topCities);

    // Get top cuisines from restaurants (real calculation)
    const cuisineStats: Record<string, number> = {};

    // Count orders by cuisine - each order counts once per cuisine
    orders.forEach((order: any) => {
      if (order.restaurant && order.restaurant.cuisines) {
        // For each order, count it once for each cuisine the restaurant offers
        order.restaurant.cuisines.forEach((cuisine: string) => {
          cuisineStats[cuisine] = (cuisineStats[cuisine] || 0) + 1;
        });
      }
    });

    // Convert to array and sort by order count
    const topCuisines = Object.entries(cuisineStats)
      .map(([cuisine, count]) => ({
        cuisine,
        orders: count,
```

```
        percentage: Math.round((count / totalOrders) * 100 * 100) / 100, // Round to 2 decimal places
      }))
      .sort((a, b) => b.orders - a.orders);
    // Removed .slice(0, 10) to show ALL cuisines

    console.log("Top cuisines:", topCuisines);

    // Get recent orders - show ALL orders within the time range, not just 5
    const recentOrders = orders
      .filter((order: any) => order.deliveryDetails)
      .sort(
        (a, b) =>
          new Date(b.createdAt).getTime() - new Date(a.createdAt).getTime()
      )
      // Removed .slice(0, 5) to show ALL orders within the time range
      .map((order: any) => ({
        id: order._id.toString(),
        customer: order.deliveryDetails.name,
        amount: order.totalAmount || 0,
        status: order.status,
        date: order.createdAt.toISOString().split("T")[0],
      }));

    console.log("Recent orders:", recentOrders);

    // Calculate monthly data
    const monthlyData = [];
    const months = [
      "Jan",
      "Feb",
      "Mar",
      "Apr",
      "May",
      "Jun",
      "Jul",
      "Aug",
      "Sep",
      "Oct",
      "Nov",
      "Dec",
    ];

    for (let i = 0; i < 12; i++) {
      const monthStart = new Date();
      monthStart.setMonth(monthStart.getMonth() - (11 - i));
      monthStart.setDate(1);

      const monthEnd = new Date(monthStart);
      monthEnd.setMonth(monthEnd.getMonth() + 1);
      monthEnd.setDate(0);

      const monthOrders = orders.filter((order: any) => {
        const orderDate = new Date(order.createdAt);
```

```
          return orderDate >= monthStart && orderDate <= monthEnd;
        });

        const monthRevenue = monthOrders.reduce((sum: number, order: any) => {
          const amount = order.totalAmount || 0;
          return sum + amount;
        }, 0);

        monthlyData.push({
          month: months[monthStart.getMonth()],
          orders: monthOrders.length,
          revenue: monthRevenue,
        });
      }

      console.log("Monthly data:", monthlyData);

      // Calculate growth by comparing current period with previous period
      const previousStartDate = new Date(startDate);
      previousStartDate.setDate(previousStartDate.getDate() - days);

      const previousOrders = await Order.find({
        createdAt: {
          $gte: previousStartDate,
          $lt: startDate,
        },
      });

      const previousTotalRevenue = previousOrders.reduce(
        (sum: number, order: any) => {
          return sum + (order.totalAmount || 0);
        },
        0
      );

      const previousTotalOrders = previousOrders.length;

      // More intelligent growth calculation
      let orderGrowth = 0;
      let revenueGrowth = 0;

      // Check if this is a new business (less than 60 days of data)
      const firstOrder = await Order.findOne().sort({ createdAt: 1 });
      const businessAge = firstOrder
        ? Math.floor(
            (new Date().getTime() - new Date(firstOrder.createdAt).getTime()) /
              (1000 * 60 * 60 * 24)
          )
        : 0;

      if (businessAge < 60) {
        // For new businesses, show month-over-month comparison instead
        const currentMonth = new Date().getMonth();
        const currentYear = new Date().getFullYear();
```

```
        const previousMonth = currentMonth === 0 ? 11 : currentMonth - 1;
        const previousYear = currentMonth === 0 ? currentYear - 1 : currentYear;

        const currentMonthOrders = await Order.find({
          createdAt: {
            $gte: new Date(currentYear, currentMonth, 1),
            $lt: new Date(currentYear, currentMonth + 1, 1),
          },
        });

        const previousMonthOrders = await Order.find({
          createdAt: {
            $gte: new Date(previousYear, previousMonth, 1),
            $lt: new Date(previousYear, previousMonth + 1, 1),
          },
        });

        const currentMonthRevenue = currentMonthOrders.reduce(
          (sum: number, order: any) => sum + (order.totalAmount || 0),
          0
        );
        const previousMonthRevenue = previousMonthOrders.reduce(
          (sum: number, order: any) => sum + (order.totalAmount || 0),
          0
        );

        if (previousMonthOrders.length === 0 && currentMonthOrders.length > 0) {
          orderGrowth = 100; // New month with orders
          revenueGrowth = 100;
        } else if (previousMonthOrders.length > 0) {
          orderGrowth = Math.round(
            ((currentMonthOrders.length - previousMonthOrders.length) /
              previousMonthOrders.length) *
              100
          );
          revenueGrowth = Math.round(
            ((currentMonthRevenue - previousMonthRevenue) /
              previousMonthRevenue) *
              100
          );
        } else {
          orderGrowth = 0;
          revenueGrowth = 0;
        }

        console.log("Month-over-month comparison:", {
          currentMonth: `${currentYear}-${currentMonth + 1}`,
          previousMonth: `${previousYear}-${previousMonth + 1}`,
          currentOrders: currentMonthOrders.length,
          previousOrders: previousMonthOrders.length,
          currentRevenue: currentMonthRevenue,
          previousRevenue: previousMonthRevenue,
        });
      } else {
```

```
      // For established businesses, use period-over-period comparison
      if (previousTotalOrders === 0 && totalOrders > 0) {
        orderGrowth = 100;
        revenueGrowth = 100;
      } else if (previousTotalOrders > 0) {
        orderGrowth = Math.round(
          ((totalOrders - previousTotalOrders) / previousTotalOrders) * 100
        );
        revenueGrowth = Math.round(
          ((totalRevenue - previousTotalRevenue) / previousTotalRevenue) * 100
        );
      } else {
        orderGrowth = 0;
        revenueGrowth = 0;
      }
    }

    console.log("Current period:", {
      startDate: startDate.toISOString(),
      endDate: new Date().toISOString(),
      orders: totalOrders,
      revenue: totalRevenue,
    });
    console.log("Previous period:", {
      startDate: previousStartDate.toISOString(),
      endDate: startDate.toISOString(),
      orders: previousTotalOrders,
      revenue: previousTotalRevenue,
    });
    console.log("Growth calculations:", { orderGrowth, revenueGrowth });

    const analyticsData = {
      totalOrders,
      totalRevenue,
      averageOrderValue,
      totalCustomers: uniqueCustomers,
      orderGrowth,
      revenueGrowth,
      topCities,
      topCuisines,
      recentOrders,
      monthlyData,
    };

    console.log("Sending analytics response:", analyticsData);
    res.json(analyticsData);
  } catch (error) {
    console.log("Analytics error:", error);
    console.log(
      "Error stack:",
      error instanceof Error ? error.stack : "No stack trace"
    );
    res.status(500).json({ message: "Error fetching analytics data" });
  }
```

```
};

export default {
  getAnalyticsData,
};

```